وزارة التعليم العالي والبحث العلمي

الجمهورية العربية السورية

الجامعة الافتراضية السورية



وظيفة البرمجة و التصميم غرضية التوجه

TIC_IPG203 _F25

إشراف الدكتور :

غيث سمير مصطفى

إعداد الطلاب :

الاسم	المعرف الجامعي	الصف
عدنان عمار الصناديقي	adnan_355470	C2
ايات باسم محمد	ayat_357473	C2
ميرفت محمد خالد طبيخ	meervat_353960	C2
محمد عيد ناصر	mohamed_eid_173444	C2
محمد محجوب	Mohammed_245077	C2

تطبيق مبادئ البرمجة غرضية التوجه (OOP) في نظام إدارة مركز صيانة الأجهزة المنزلية

مقدمة

تم تطوير نظام إدارة مركز صيانة الأجهزة المنزلية باستخدام لغة البرمجةC# ، بهدف إدارة الأجهزة التي تحتاج إلى الصيانة، مثل الثلاجات والغسالات والمكيفات، ومتابعة حالتها وتكلفة إصلاحها وموعد تسليمها. يعتمد النظام على مبادئ البرمجة غرضية التوجه(Object-Oriented Programming) ، وهي التجريد والتغليف والوراثة وتعدد الأشكال، بالإضافة إلى استخدام التفويض والأحداث والفئات والأعضاء الثابتة.

أولاً: التجريد(Abstraction)

تم تطبيق مبدأ التجريد من خلال إنشاء واجهة (Interface) باسم `IRepairable`، وفئة مجردة (Abstract Class)باسم `Appliance`

تحتوي الواجهة `IRepairable` على الدالتين التاليتين:

- `GetStatus()`: تستخدم للحصول على حالة الجهاز الحالية.
- `CalculateRepairCost()`: تستخدم لحساب تكلفة إصلاح الجهاز.

تهدف الواجهة إلى تحديد العمليات الأساسية التي يجب أن تتوفر في الأجهزة القابلة للصيانة، دون تحديد كيفية تنفيذ هذه العمليات.

أما الفئة المجردة `Appliance`، فتطبق الواجهة `IRepairable`وتحتوي على الخصائص المشتركة بين الأجهزة، مثل `SerialNumber`و `Brand`و `Status`و `DeliveryDate` كما تحتوي على الدالة المجردة `CalculateRepairCost()`، التي يجب على كل فئة فرعية تنفيذها بطريقة تناسب نوع الجهاز.

يساعد هذا المبدأ على توحيد العمليات المشتركة بين جميع الأجهزة، وتقليل تكرار الكود، وتنظيم تصميم النظام.

ثانياً: الوراثة(Inheritance)

تم تطبيق مبدأ الوراثة من خلال إنشاء ثلاث فئات فرعية ترث من الفئة المجردة `Appliance`، وهي:

1. الفئة `Refrigerator`التي تمثل الثلاجات.
2. الفئة `WashingMachine`التي تمثل الغسالات.
3. الفئة `AirConditioner`التي تمثل المكيفات.

ترث هذه الفئات الخصائص والدوال المشتركة من الفئة `Appliance`، مثل `SerialNumber` و `Brand`و `GetStatus()`، دون الحاجة إلى إعادة كتابتها في كل فئة.

كما تعيد كل فئة تعريف الدالة `CalculateRepairCost()` باستخدام الكلمة `override`، بحيث تكون تكلفة إصلاح الثلاجة 150، والغسالة 100، والمكيف 200، وفقاً للقيم الافتراضية المستخدمة في البرنامج.

يساعد هذا المبدأ على إعادة استخدام الكود، وتسهيل إضافة أنواع جديدة من الأجهزة مستقبلاً.

ثالثاً: تعدد الأشكال(Polymorphism)

تم تطبيق تعدد الأشكال من خلال الدالة `CalculateRepairCost()` الموجودة في الفئة المجردة `Appliance`، والتي تعيد الفئات الفرعية تعريفها وفقاً لنوع الجهاز.

يظهر هذا المبدأ عند استخدام قائمة من النوع `List<Appliance>` داخل الفئة `RepairCenter`، حيث يمكن للقائمة أن تحتوي على كائنات من الفئات `Refrigerator` و`WashingMachine` و`AirConditioner`

عند استدعاء الدالة `CalculateRepairCost()` على كل كائن داخل القائمة، يتم تنفيذ النسخة المناسبة لنوع الجهاز الفعلي.

كما تم استخدام تعدد الأشكال في الدالة `CalculateTotalCost()` داخل الفئة `RepairCenter`، حيث تجمع تكاليف إصلاح جميع الأجهزة دون الحاجة إلى كتابة شرط منفصل لكل نوع.

يساعد تعدد الأشكال على جعل الكود أكثر مرونة، ويسمح بإضافة أنواع جديدة من الأجهزة دون الحاجة إلى تعديل طريقة حساب التكلفة الإجمالية.

رابعاً: التغليف(Encapsulation)

تم تطبيق مبدأ التغليف داخل الفئة `Appliance` من خلال جعل الحقول خاصة باستخدام الكلمة `private`، مثل:

- `serialNumber`: لتخزين الرقم التسلسلي للجهاز
- `brand`: لتخزين العلامة التجارية
- `status`: لتخزين حالة الجهاز
- `deliveryDate`: لتخزين موعد التسليم المتوقع

تم توفير خصائص للوصول إلى هذه البيانات، وهي `SerialNumber` و`Brand` و`Status` و`DeliveryDate`

تسمح هذه الخصائص بقراءة البيانات، بينما لا تسمح بتغيير الرقم التسلسلي والعلامة التجارية وموعد التسليم بعد إنشاء الكائن، لأنها لا تحتوي على أدوات ضبط عامة(Public Setters)

أما تغيير حالة الجهاز، فيتم من خلال الدالة ChangeStatus()، التي تتحقق من أن الحالة الجديدة واحدة من الحالات المسموح بها، وهي Under Maintenanceأو Repairedأو Delivered

كذلك يتحقق المنشئ من صحة الرقم التسلسلي، ويتأكد من أن العلامة التجارية ليست فارغة.

يساعد التغليف على حماية بيانات الأجهزة، ومنع التعديلات غير الصحيحة، وضمان تغيير الحالة من خلال آلية محددة داخل النظام.

خامساً: التفويض والأحداث(Delegates & Events)

تم استخدام التفويض والأحداث لتوفير آلية إشعارات تلقائية عند حدوث تغييرات مهمة في حالة الأجهزة.

1- التفويض(Delegate)

تم تعريف التفويض باسم StatusChangedHandler، وهو يحدد شكل الدالة التي تستقبل رقم الجهاز التسلسلي ورسالة الإشعار.

يسمح التفويض بربط الحدث بدالة مناسبة تستقبل البيانات المطلوبة وتتعامل معها.

2- الأحداث(Events)

تم تعريف الحدث StatusChangedداخل الفئة Appliance ويُستخدم لإشعار الأجزاء الأخرى من النظام عند حدوث تغيير مهم.

تحتوي الفئة Applianceعلى الدالة OnStatusChanged()، التي تطلق الحدث إذا كانت هناك دالة مشتركة به.

يتم استدعاء هذه الدالة عند تغيير حالة الجهاز من خلال ChangeStatus()، مثل الانتقال إلى حالة Repairedعند اكتمال الإصلاح، أو إلى حالة Deliveredعند تسليم الجهاز.

كما تستخدم الدالة CheckDeliveryDate()للتحقق من تأخر التسليم. فإذا تجاوز التاريخ الحالي موعد التسليم المتوقع وكان الجهاز ما يزال قيد الصيانة، يتم إطلاق إشعار تحذيري.

داخل الفئة RepairCenter، يتم الاشتراك بالحدث من خلال السطر:

```
appliance.StatusChanged += ShowNotification;
```

وتتولى الدالة ShowNotification()عرض رقم الجهاز ورسالة الإشعار على الشاشة.

دور التفويض والأحداث في النظام

يتيح التفويض تحديد شكل الدالة المسؤولة عن استقبال الإشعار، بينما يسمح الحدث بإبلاغ مركز الصيانة عند حدوث تغيير معين دون الحاجة إلى استدعاء دالة الإشعارات مباشرة من كل جزء في البرنامج.

فعلى سبيل المثال، عند اكتمال إصلاح الثلاجة، يتم إطلاق الحدث، ثم تستقبل الفئة `RepairCenter` الإشعار وتعرض رسالة تفيد باكتمال الإصلاح. وبالمثل، يمكن عرض تحذير عند تأخر تسليم جهاز.

يساعد هذا الأسلوب على فصل عملية تغيير حالة الجهاز عن عملية عرض الإشعارات، مما يجعل النظام أكثر تنظيماً وقابلية للتطوير.

سادساً: الفئات والأعضاء الثابتة(Static Classes & Members)

تم تطبيق هذا المبدأ باستخدام الفئة الثابتة `DataValidator`، بالإضافة إلى دوال ثابتة داخل الفئة `RepairCenter`

1- الفئة الثابتة DataValidator

تحتوي الفئة `DataValidator`على دوال للتحقق من صحة البيانات، وهي:

- `IsValidSerialNumber()`: تتحقق من أن الرقم التسلسلي غير فارغ وأن طوله لا يقل عن ثلاثة أحرف
- `IsValidDate()`: تتحقق من أن التاريخ ليس القيمة الافتراضية `DateTime.MinValue`

تُستدعى هذه الدوال مباشرة باستخدام اسم الفئة `DataValidator`، دون الحاجة إلى إنشاء كائن منها.

2 - الأعضاء الثابتة داخل RepairCenter

تحتوي الفئة `RepairCenter`على الدالتين الثابتتين التاليتين:

- `CountByStatus()`: تحسب عدد الأجهزة التي تحمل حالة محددة، مثل الأجهزة التي ما تزال قيد الصيانة أو التي تم إصلاحها
- `CalculateTotalCost()`: تحسب مجموع تكاليف إصلاح جميع الأجهزة الموجودة في القائمة

تساعد هذه الدوال على تنفيذ عمليات التحقق والإحصاء والحساب بطريقة منظمة، وإعادة استخدامها دون الحاجة إلى إنشاء كائن من الفئة التي تحتوي عليها إذا كانت الدالة ثابتة.

سابعاً: دور الفئة RepairCenter في النظام

تُعد الفئة `RepairCenter`الفئة المسؤولة عن إدارة الأجهزة داخل مركز الصيانة. وتحتوي على قائمة خاصة باسم `appliances`لتخزين الأجهزة المسجلة.

وتتضمن أهم الدوال المستخدمة فيها:

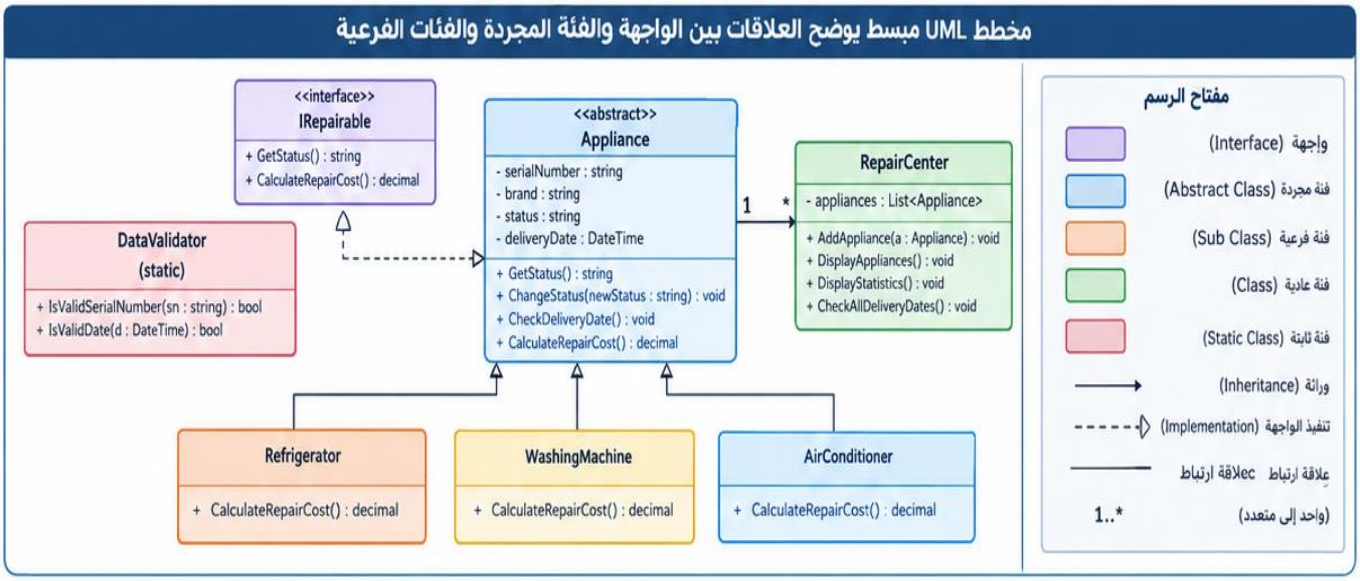
- `AddAppliance()`: لإضافة جهاز جديد إلى المركز، مع منع تكرار الرقم التسلسلي والاشتراك بحدث تغير الحالة
- `DisplayAppliances()`: لعرض بيانات الأجهزة وحالاتها وتكاليف إصلاحها
- `ShowNotification()`: لاستقبال إشعارات الأحداث وعرضها
- `DisplayStatistics()`: لعرض إحصائيات المركز، بما فيها عدد الأجهزة حسب الحالة وإجمالي تكاليف الإصلاح
- `CheckAllDeliveryDates()`: لفحص مواعيد تسليم جميع الأجهزة واكتشاف الأجهزة المتأخرة
- `GetAppliances()`: لإرجاع نسخة من قائمة الأجهزة المسجلة

تربط هذه الفئة بين الكائنات المختلفة وتوفر العمليات اللازمة لإدارة الأجهزة ومتابعة حالتها.

جدول ملخص مبادئ OOP في نظام إدارة مركز الصيانة :

المبدأ	كيفية تطبيقه في المشروع	أسماء الفئات والدوال والخصائص
التجريد (Abstraction)	تحديد العمليات المشتركة بين الأجهزة وإخفاء تفاصيل تنفيذها.	IRepairable، Appliance، GetStatus() CalculateRepairCost()
الوراثة (Inheritance)	إنشاء فئات للأجهزة ترث الخصائص والدوال المشتركة من الفئة الأساسية.	Appliance، Refrigerator AirConditioner، WashingMachine
تعدد الأشكال (Polymorphism)	تنفيذ دالة حساب تكلفة الإصلاح بصورة مختلفة حسب نوع الجهاز.	CalculateRepairCost()، override CalculateTotalCost()
التغليف (Encapsulation)	حماية بيانات الأجهزة باستخدام الحقول الخاصة والخصائص، والتحكم بتغيير الحالة.	serialNumber، brand، status SerialNumber، Brand، Status ChangeStatus()
التفويض (Delegate)	تحديد شكل الدالة التي تستقبل إشعارات تغير حالة الجهاز.	StatusChangedHandler
الأحداث (Events)	إرسال إشعارات عند اكتمال الإصلاح أو تسليم الجهاز أو تأخر التسليم.	StatusChanged، OnStatusChanged() ShowNotification()، CheckDeliveryDate()
الفئات الثابتة (Static Classes)	توفير دوال للتحقق من صحة البيانات دون إنشاء كائن من الفئة.	DataValidator، IsValidSerialNumber() IsValidDate()
الأعضاء الثابتة (Static Members)	حساب عدد الأجهزة حسب الحالة وإجمالي تكاليف الإصلاح.	CountByStatus()، CalculateTotalCost() داخل RepairCenter

مخطط uml بسيط تم تصميمه على drow.io :



الكود البرمجي الكامل جاهز للتنفيذ :

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace HomeApplianceRepairCenter
{
    // 1. Abstraction: المشتركة العمليات واجهة
    public interface IRepairable
    {
        string GetStatus();
        decimal CalculateRepairCost();
    }

    // الحالة تغير بإشعارات الخاص التفويض تعريف
    public delegate void StatusChangedHandler(
        string serialNumber,
        string message
    );

    // 2. المجردة الأساسية الفئة
    public abstract class Appliance : IRepairable
    {
        // التغليف لتحقيق خاصة حقول
        private readonly string serialNumber;
        private readonly string brand;
        private string status;
        private readonly DateTime deliveryDate;

        // الجهاز حالة بتغير الخاص الحدث
        public event StatusChangedHandler StatusChanged;

        // فقط للقراءة خصائص
        public string SerialNumber
```

```
        {
            get { return serialNumber; }
        }

        public string Brand
        {
            get { return brand; }
        }

        public string Status
        {
            get { return status; }
        }

        public DateTime DeliveryDate
        {
            get { return deliveryDate; }
        }

        // المنشئ
        protected Appliance(
            string serialNumber,
            string brand,
            DateTime deliveryDate)
        {
            if (!DataValidator.IsValidSerialNumber(serialNumber))
            {
                throw new ArgumentException(
                    "Invalid serial number"
                );
            }

            if (string.IsNullOrWhiteSpace(brand))
            {
                throw new ArgumentException(
                    "Brand cannot be empty"
                );
            }

            this.serialNumber = serialNumber;
            this.brand = brand;
            this.deliveryDate = deliveryDate;

            // للجهاز الافتراضية الحالة
            this.status = "Under Maintenance";
        }

        // الجهاز حالة إرجاع
        public string GetStatus()
        {
            return status;
        }

        // الإصلاح تكلفة تحسب مجردة دالة
        public abstract decimal CalculateRepairCost();

        // الجهاز حالة تغيير
        public void ChangeStatus(string newStatus)
        {
```

```
        if (newStatus != "Under Maintenance" &&
            newStatus != "Repaired" &&
            newStatus != "Delivered")
        {
            Console.WriteLine("Invalid status!");
            return;
        }

        if (status == newStatus)
        {
            return;
        }

        status = newStatus;

        if (newStatus == "Repaired")
        {
            OnStatusChanged(
                "Repair completed successfully."
            );
        }
        else if (newStatus == "Delivered")
        {
            OnStatusChanged(
                "Appliance delivered successfully."
            );
        }
        else
        {
            OnStatusChanged(
                "Appliance returned to maintenance."
            );
        }
    }

    // التسليم موعد عن التأخر فحص
    public void CheckDeliveryDate()
    {
        if (DateTime.Today > deliveryDate.Date &&
            status == "Under Maintenance")
        {
            OnStatusChanged(
                "WARNING: Delivery is overdue!"
            );
        }
    }

    // مع متوافقة بطريقة الحدث إطلاق C# 5
    protected virtual void OnStatusChanged(string message)
    {
        if (StatusChanged != null)
        {
            StatusChanged(serialNumber, message);
        }
    }
}

// 3. المشتقة الفئة :الثلاجة
public class Refrigerator : Appliance
```

```
{
    public Refrigerator(
        string serialNumber,
        string brand,
        DateTime deliveryDate)
        : base(serialNumber, brand, deliveryDate)
    {
    }

    public override decimal CalculateRepairCost()
    {
        return 150m;
    }
}

// الفئة المشتقة: الغسالة
public class WashingMachine : Appliance
{
    public WashingMachine(
        string serialNumber,
        string brand,
        DateTime deliveryDate)
        : base(serialNumber, brand, deliveryDate)
    {
    }

    public override decimal CalculateRepairCost()
    {
        return 100m;
    }
}

// الفئة المشتقة: المكيف
public class AirConditioner : Appliance
{
    public AirConditioner(
        string serialNumber,
        string brand,
        DateTime deliveryDate)
        : base(serialNumber, brand, deliveryDate)
    {
    }

    public override decimal CalculateRepairCost()
    {
        return 200m;
    }
}

// 4. فئة ثابتة للتحقق من صحة البيانات
public static class DataValidator
{
    public static bool IsValidSerialNumber(
        string serialNumber)
    {
        return !string.IsNullOrWhiteSpace(serialNumber)
            && serialNumber.Length >= 3;
    }
```

```
    public static bool IsValidDate(DateTime date)
    {
        return date != DateTime.MinValue;
    }
}

// 5. الصيانة مركز إدارة عن المسؤولة الفئة
public class RepairCenter
{
    // الأجهزة قائمة
    private List<Appliance> appliances;

    public RepairCenter()
    {
        appliances = new List<Appliance>();
    }

    // المركز إلى جهاز إضافة
    public void AddAppliance(Appliance appliance)
    {
        if (appliance == null)
        {
            Console.WriteLine("Invalid appliance!");
            return;
        }

        // التسلسلي الرقم تكرار منع
        foreach (Appliance item in appliances)
        {
            if (item.SerialNumber == appliance.SerialNumber)
            {
                Console.WriteLine(
                    "Serial number already exists!"
                );
                return;
            }
        }

        // بالحدث الاشتراك
        appliance.StatusChanged += ShowNotification;

        appliances.Add(appliance);
    }

    // الأجهزة إشعارات تستقبل التي الدالة
    private void ShowNotification(
        string serialNumber,
        string message)
    {
        Console.WriteLine();
        Console.WriteLine("===== NOTIFICATION =====");
        Console.WriteLine("Serial Number: " + serialNumber);
        Console.WriteLine("Message: " + message);
        Console.WriteLine("========================");
        Console.WriteLine();
    }

    // الأجهزة جميع عرض
    public void DisplayAppliances()
```

```
    {
        Console.WriteLine();
        Console.WriteLine("===== APPLIANCE LIST =====");

        foreach (Appliance appliance in appliances)
        {
            Console.WriteLine(
                "Type: " + appliance.GetType().Name
            );

            Console.WriteLine(
                "Serial Number: " + appliance.SerialNumber
            );

            Console.WriteLine(
                "Brand: " + appliance.Brand
            );

            Console.WriteLine(
                "Status: " + appliance.GetStatus()
            );

            Console.WriteLine(
                "Delivery Date: " +
                appliance.DeliveryDate.ToShortDateString()
            );

            Console.WriteLine(
                "Repair Cost: " +
                appliance.CalculateRepairCost()
            );

            Console.WriteLine("-------------------------");
        }
    }

    // الحالة حسب الأجهزة عدد لحساب ثابت عضو
    public static int CountByStatus(
        List<Appliance> applianceList,
        string requiredStatus)
    {
        int count = 0;

        foreach (Appliance appliance in applianceList)
        {
            if (appliance.Status == requiredStatus)
            {
                count++;
            }
        }

        return count;
    }

    // الإجمالية التكلفة لحساب ثابت عضو
    public static decimal CalculateTotalCost(
        List<Appliance> applianceList)
    {
        decimal total = 0;
```

```
            foreach (Appliance appliance in applianceList)
            {
                total += appliance.CalculateRepairCost();
            }

            return total;
        }

        // الأجهزة قائمة إعادة
        public List<Appliance> GetAppliances()
        {
            return new List<Appliance>(appliances);
        }

        // المتأخرة الأجهزة جميع فحص
        public void CheckAllDeliveryDates()
        {
            foreach (Appliance appliance in appliances)
            {
                appliance.CheckDeliveryDate();
            }
        }

        // المركز إحصائيات عرض
        public void DisplayStatistics()
        {
            Console.WriteLine();
            Console.WriteLine("===== REPAIR CENTER STATISTICS =====");

            Console.WriteLine(
                "Total Appliances: " + appliances.Count
            );

            Console.WriteLine(
                "Under Maintenance: " +
                CountByStatus(
                    appliances,
                    "Under Maintenance"
                )
            );

            Console.WriteLine(
                "Repaired: " +
                CountByStatus(appliances, "Repaired")
            );

            Console.WriteLine(
                "Delivered: " +
                CountByStatus(appliances, "Delivered")
            );

            Console.WriteLine(
                "Total Repair Cost: " +
                CalculateTotalCost(appliances)
            );
        }
    }
```

```
// 6. البرنامج الرئيسي
class Program
{
    static void Main(string[] args)
    {
        // إنشاء مركز الصيانة
        RepairCenter center = new RepairCenter();

        // إنشاء ثلاثة أجهزة
        Refrigerator refrigerator = new Refrigerator(
            "R001",
            "Samsung",
            DateTime.Today.AddDays(2)
        );

        WashingMachine washingMachine = new WashingMachine(
            "W001",
            "LG",
            DateTime.Today.AddDays(3)
        );

        AirConditioner airConditioner = new AirConditioner(
            "A001",
            "Gree",
            DateTime.Today.AddDays(-1)
        );

        // إضافة الأجهزة إلى المركز
        center.AddAppliance(refrigerator);
        center.AddAppliance(washingMachine);
        center.AddAppliance(airConditioner);

        // عرض الأجهزة
        center.DisplayAppliances();

        // تغيير حالة الثلاجة إلى الإصلاح تم
        refrigerator.ChangeStatus("Repaired");

        // تسليم الثلاجة
        refrigerator.ChangeStatus("Delivered");

        // فحص الأجهزة المتأخرة
        center.CheckAllDeliveryDates();

        // عرض الإحصائيات
        center.DisplayStatistics();

        Console.WriteLine();
        Console.WriteLine("Press any key to exit...");
        Console.ReadKey();
    }
}
}
```

رابط المشروع على **Github** :